

第二十一届全国大学生智能汽车竞赛

技 术 报 告

学 校： 哈尔滨工程大学

队伍名称： 济海追风 7 队

参赛队员： 王绍旭
 湛铭宇
 王千然

带队教师： 张立伟

关于技术报告和研究论文使用授权的说明

本人完全了解全国大学生智能汽车竞赛关保留、使用技术报告和研究论文的规定，即：参赛作品著作权归参赛者本人，比赛组委会可以在相关主页上收录并公开参赛作品的设计方案、技术报告以及参赛模型车的视频、图像资料，并将相关内容编纂收录在组委会出版论文集中。

参赛队员签名：

王绍旭

潘铭宇

王千然

带队教师签名：

张宏伟

日期：2026年

8月16日

摘 要

本文设计的智能车系统以 TC264 微控制器为核心控制单元，通过陀螺仪获取车体当前姿态，用编码器检测模型车的实时速度，主要使用串级 PID 控制算法使车稳定前行，以及作为后续跟踪的基础。为了加快对模型车的参数调试过程，为调参提供方向，使用了多种调试工具，进行了大量硬件与软件测试。实验结果表明，该系统设计方案确实可行。

目 录

摘要.....	I
引 言.....	IV
第一章 方案设计.....	1
1.1 系统总体方案的设计.....	1
第二章 智能车机械结构调整与优化.....	2
2.1 智能车车体机械建模.....	2
2.2 智能车车体机械结构与调整.....	3
2.2.1 转向几何与前叉设计.....	3
2.2.2 车架与质量分配.....	4
2.2.3 转动惯量.....	4
2.3 智能车防撞结构的调整.....	5
2.4 对智能车重心的调整.....	5
2.5 舵机与驱动系统.....	6
2.6 轮胎的保养与使用.....	6
2.7 IMU 的安装.....	7
2.8 其他器件的固定和保护策略.....	8
第三章 硬件电路设计说明.....	9
3.1 总体设计原则与硬件架构拓扑.....	9
3.1.1 总体设计原则.....	9
3.1.2 系统供电与信号拓扑结构.....	9
3.2 智能车主控与核心感知电路设计.....	10
3.2.1 TC264 主控核心电路.....	10
3.2.2 IMU660RA 姿态传感器电路与点胶防噪防护.....	11
3.2.3 TLE5012BE1000 磁编码器电路.....	11
3.3 电源管理与安全防护电路设计.....	13
3.3.1 多级稳压拓扑.....	13
3.3.2 电池防反接与保护电路.....	13
3.4 三相无刷电机驱动电路设计.....	14
3.4.1 CYT2BL3 无刷驱动控制主控与 RY9121E 电源.....	14
3.4.2 三相逆变功率半桥与 PCB 散热设计.....	15

3.5 人机交互与外设扩展接口电路.....	16
3.5.1 显示屏与无线调试串口接口.....	16
3.5.2 五向按键与蜂鸣器提示电路.....	16
3.6 PCB 布局布线与 EMC 抗干扰优化.....	17
第四章 智能车控制软件设计说明.....	19
4.1 路径生成与跟踪.....	19
4.1.1 路径几何生成.....	19
4.1.2 航向推算.....	21
4.1.3 跟踪点选择.....	22
4.1.4 Stanley 航向控制.....	23
4.2 PID 控制算法介绍.....	24
4.2.1 位置式 PID.....	26
4.2.2 增量式 PID.....	27
4.2.3 PID 参数整定.....	28
4.3 串级 PID 的设计.....	28
4.4 Mahony 算法.....	29
第五章 开发工具、制作、安装、调试过程说明.....	31
5.1 开发工具.....	31
5.2 调试过程简介.....	31
结 论.....	34
附录：程序源代码.....	35

引言

全国大学生智能汽车竞赛是以“立足培养、重在参与、鼓励探索、追求卓越”为宗旨，鼓励创新的一项科技竞赛活动。竞赛要求在规定的汽车模型平台上，使用恩智浦半导体公司的微控制器作为核心控制模块，通过增加姿态传感器、电机驱动模块以及编写相应控制程序，制作完成一个能够自主识别道路模型汽车。参赛队员的目标是模型车需要按照规则，慢速时用时最长且平稳运行，快速时用时最短到达终点。

在本次比赛中，本组使用竞赛车 N1 型车模，采用微控制器 TC264 作为核心控制单元，自主构思控制方案及系统设计，融合陀螺仪角度采集和计算、编码器获取速度、电机驱动输出，最终实现一套能够跟踪轨迹的控制系统。

在制作小车的过程中，我们对小车的整体构架进行了深入的研究，分别在机械结构、硬件和软件上都进行过改进，硬件上主要是考虑并实践各种传感器的布局，改进驱动电路，软件上先后进行了几次大改，路径跟踪采用基于纯惯导的航位推算与轨迹控制方法。控制算法上，重点研究了串级 PID 控制结构，外环为航向与横向位置控制，内环为角速度控制，对速度闭环和姿态闭环也进行了一些研究。

在这份报告中，我们主要通过对整体方案、机械、硬件、算法等方面的介绍，详细阐述我队在此次智能汽车竞赛中的思想和创新。具体表现在电路的创新设计、算法以及辅助调试模块等方面的创新。我队成员涉及自动化、机械、计算机等专业，在准备比赛的过程中，队员查阅了大量的专业资料，反复地调试智能车模型的各项参数。所有队员都为此次智能汽车竞赛付出了艰苦的劳动。

第一章 方案设计

1.1 系统总体方案的设计

单车的 TC264 微处理器通过采集陀螺仪的模拟信号，得到车身倾角姿态的状态数据；通过采集磁编码器对车轮转速的脉冲计数，得到车行进的速度数据。通过微处理器对角度、速度进行 PID 控制，最后以 PWM 波输出驱动电机。在调试过程中，通过车身姿态角度、车速等各类实时数据，便于查看车的状态情况，进行进一步的调试。

第二章 智能车机械结构调整与优化

智能车系统的控制性能不仅在于控制算法的合理、高效，也建立在其机械结构的设计。因此在设计控制架构和算法逻辑之前，有必要理解车模结构的特性，建立合适的运动学和动力学模型。针对实际情况和需求，来调整机械结构，并需结合具体的控制策略对机械结构进行针对性调整。在实际调试的时候出现问题时，考虑对机械结构进行优化。本章将主要介绍智能车车模的机械结构和调整方案。

2.1 智能车车体机械建模

此次比赛选用的单车车模采用 N1 车模，赛车机械结构仅使用竞赛提供车模的底盘、转向及驱动部分。前轮负责转向，后轮负责驱动。整车依靠后轮驱动行进，并通过前轮转向与车身动态倾角来实现循迹。不同于的四轮车模，单车车模需要依靠不断地调整姿态，凭借离心力来实现平衡，进而平稳前进的。

因此，我们对于机械结构的要求是：重心低、布局紧凑、响应灵敏。我们在不断的测试和调整确定了以下的设计方案：

表 2.1 N1 模型车基本参数

	尺寸
轴距	205mm
车轮直径	64mm
车轮宽度	13mm
总长	297mm
总宽	50mm
总高	70mm
总重	330g

车体材质为铝合金，刚性强且轻量化，上述参数为后续动力学建模与控制算

法设计提供了物理基础。

2.2 智能车车体机械结构与调整

单车在行驶中依靠动态平衡维持直立，在静止状态下不具备自稳定能力，是一个不稳定系统，类似于倒立摆。其前轮定位参数不仅影响转向性能，更直接影响平衡控制的难度与稳定性。

2.2.1 转向几何与前叉设计

单车的转向通过前轮绕前叉轴的转动实现。前叉轴的倾角设定直接影响转向特性。与四轮车类似，前叉轴在侧向平面内与垂线的夹角称为主销内倾角，在纵向平面内与垂线的夹角称为主销后倾角。

主销后倾角的作用机制是，当车轮向前滚动时，地面作用于轮胎接地点上的阻力不通过主销轴线，从而产生一个使前轮绕主销旋转的力矩。该力矩的方向总是趋于使前轮指向行进方向，这就是自动回正力矩的来源。后倾角越大，回正力矩越强，直线行驶稳定性越好，但转向时舵机需要克服的力矩也越大。

主销内倾角则使前轮在转向时产生一个抬升车头的趋势，转向时前轮绕

倾斜的主销转动，车轮接地点会相对于地面下沉或抬升，由此产生的重力分量形成回正力矩。内倾角的主要作用是减小转向力臂，使转向更轻便，同时提供回正效果。

两者虽都能提供回正力矩，但来源不同。后倾角的回正力矩与车速成正比，车速越高，则回正越强。而内倾角的回正力矩与车速无关，仅取决于转向角度。对于单车，过大的倾角会增加舵机负载，降低转向响应速度，因此转向通常控制在不宜过大的角度，使舵机能更好地执行转向指令。

此外，前叉的刚度也至关重要。前叉在侧向力作用下若发生弹性变形，会导致实际前轮转角与舵机指令之间的偏差，产生转向延迟。因此，前叉应采用刚性材料制作，并确保与车架的连接无虚位。

2.2.2 车架与质量分配

车架是承载所有器件的基体，其结构形式决定了整车质量的分布特征。对于单车而言，重心高度是最敏感的参数。由刚体动力学可知，车体绕接触点（轮胎接地点）的倾覆力矩与重心高度成正比。重心越高，维持平衡所需的侧向加速度就越大，舵机需要提供的转角补偿也越大，系统的稳定裕度随之降低。因此，在结构设计上应尽可能将所有器件贴底布置，使重心趋近于驱动轴轴线高度。

重心的前后位置同样重要。从纵向受力来看，后轮为驱动轮，提供前进推力；前轮为转向轮，提供侧向力。重心靠后，则后轮垂直载荷增大，有利于驱动力的传递，但前轮载荷减轻会削弱转向响应；重心靠前，则转向灵敏度提升，但后轮可能因载荷不足而在加速时打滑。合理的质量分配应使前后轮在静态时均有一定的正压力，以保证驱动与转向各自具备足够的附着条件。

此外，车体的左右对称性在后续控制算法设计过程中是不可忽视的。左右质量不对称会在直线行驶时产生偏航力矩，但实际上无法真正做到整车左右质量分布对称，所以在控制代码中会加入机械零点偏移，从软件方面减小这方面对控制的干扰。

2.2.3 转动惯量

单车在动态平衡过程中，车体需要做快速的倾斜摆动。由转动惯量的

定义可知，质量分布离转动轴越远，转动惯量越大。过大的转动惯量意味着车体对舵机指令的角加速度响应变慢，控制系统的带宽受到限制。因此，在结构设计上应尽量使质量集中在车体中心附近。

2.3 智能车防撞结构的调整

单车车模的底盘设有一根横向杆，位于车体中段。这根杆的首要功能是防撞保护。在车辆失控侧翻或冲出赛道时，横向杆率先接触地面，以防电路板、IMU 等核心器件损坏。在最初的设计中，我们使用的是黄铜杆配合两端螺母固定，然而，在调试的时候频繁撞击，导致杆端弯曲变形或连接处固定松脱，于是改用整根碳纤维杆贯通固定的方案。碳纤维的弹性模量较高，在受到冲击时变形小。也能够将撞击能量传递至两侧支架并耗散掉，且不易发生塑性变形，同时也减轻了整车质量。

此外，在车前身的最前端加装了一个塑料 U 型环作为防撞头。该 U 型环通过两颗螺丝与车架前端的预留安装孔紧固连接。U 型环的弧形结构有利于将撞击力沿弧面分散，而不是集中在某一个点。此外，塑料材质使得当撞击力超过弹性极限时，U 型环会先于车架发生破坏，从而消耗掉多余的能量。

2.4 对智能车重心的调整

对于难以平衡的单车，通过调整车体的质量分布可以优化动态平衡特性。在车身两侧的底盘附近加装了铁质配重块，相当于将一部分质量下移并向外侧扩展。重心低，车体在发生侧倾时车体抵抗侧翻的能力会得到增强。

配重块对侧倾转动惯量起到一定调节作用。可以理解为车体绕前进方向转动的惯性，车体不容易被扰动。对于单车的动态平衡而言，陀螺仪和舵机构成的反馈控制回路存在响应带宽限制：如果车体侧倾方向的转动惯量过小，车体就过于“敏感”，行驶纠偏时容易激起高频摇摆，引发震荡，难以达到稳定姿态。适当增大侧倾转动惯量后，也有利于 PID 等控制器的参数整定和稳定运行



图 2.1 配重块的安装

2.5 舵机与驱动系统

舵机经传动机构驱动前轮绕转向轴偏转，带动前轮实现转向，传动链通常采用螺钉连接。螺钉的松紧程度需要注意。过紧的螺钉会对传动产生阻碍。由于舵机输出力矩有限，小角度转向指令可能无法驱动前轮偏转。舵机盘的旋转中心与转向轴的旋转中心应该在同一平面内。

后轮驱动系统的传动问题主要集中在齿轮啮合和轴系对上。电机输出的转矩通过减速齿轮传递至后轮，齿轮间隙过大时，加减速切换瞬间存在动力传递空程，表现为车体在油门变化时出现顿挫感。后轮电机驱动在长期使用中，电机固定螺丝同样需要定期检查，电机松动会导致齿轮啮合状态改变，产生异响。

在紧固螺丝时，使用螺丝刀或靠手感来精细控制。在刚好拧紧的同时，再略微旋转一定角度即可，不宜过紧。具体效果应该结合舵机响应情况和实际跑车效果来确定。但不论如何要做到传动顺畅。

2.6 轮胎的保养与使用

轮胎与地面直接接触，其状态直接影响附着条件和操控手感。在实际

竞赛中，轮胎性能的衰退往往是渐进且不易察觉的，因此对轮胎的日常保养和长期使用管理需要给予足够重视。

长期的户外调试，轮胎与地面持续摩擦，使得轮胎逐渐老化。老化的轮胎附着系数明显降低，尤其在低速弯道和急加速时，侧滑趋势加剧。因此，轮胎应定期检查并更换。

橡胶与轮毂的固定也是一个需要关注的环节。在调试过程中，发现在高速转弯或来回转向时，轮胎可能在轮毂上发生相对滑转。这种滑转增加了车体扰动的可能，使得车体不稳。因此需要对橡胶与轮毂之间进行处理。我们在轮毂中的沟槽中涂抹了胶水，进行粘胶处理。打胶固定的关键在于均匀涂抹。胶量过少则粘结强度不足，在后续的使用中仍会松脱。

轮胎的保养与固定工作也应值得进行。前期调试的过程中由于忽视了保养，性能飘忽不定，之后认识到轮胎保养的重要性之后，进行了轮胎保养与固定工作，车模的性能恢复了稳定。

2.7 IMU 的安装

智能车之所以能实现自平衡，核心在于 IMU 实时反馈车体的角速度与角度信息。控制算法依据 IMU 测得的数据计算出维持平衡所需的转向角度，因此 IMU 的安装位置和方式直接决定了控制质量的优劣。

从运动学原理来看，IMU 测得的角速度是相对于其自身安装位置的局部角速度。当车体发生侧倾或偏航转动时，如果 IMU 偏离车体物理重心，它会额外感受到因旋转半径带来的向心加速度分量，这一分量会被加速度计拾取并叠加在重力加速度上，使得姿态角解算产生偏差。因此，IMU 应尽可能靠近整车物理重心安装，以减小旋转运动带来的附加加速度干扰。当然，实际安装时由于电路板设计，IMU 是偏离车轴一段距离的。此外，IMU 必须在电路板固定好，为此我们进行了打胶处理。如果连接松动，在车体运行时 IMU 的测量会引入噪声，污染测量数据，有干扰控制稳定的风险。



图 2.2 IMU 的安装与固定

2.8 其他器件的固定和保护策略

对于电池这种较重的部件，其固定的可靠性直接影响整车的安全性和重心的一致性。电池固定需要满足两个要求：一是牢固，在急加速、急刹车和剧烈转向时电池不能产生任何位移；二是可调，便于在调试阶段前后移动电池以微调重心位置。

在调试阶段，我们使用可拉伸卡扣的固定环或魔术贴来固定电池。这种固定方式的优点是拆装快捷，便于频繁更换电池和调整电池位置。然而，受限于底盘电池槽的空间情况，它们无法有效压紧电池，且仅仅靠静摩擦力难以在车体加速度剧烈改变的时候稳定住电池的位置。在正式比赛时改用胶带直接粘贴固定的方式，贴合面积大、粘附力强，电池在激烈运动中不会产生很大的相对位移。同时，胶带本身具有一定的柔韧性，能适应电池外形的微小起伏，确保压紧力均匀分布。

另外，在调试的时候，为了保护车的前身，防止与坚硬地面碰撞变形，曾选择在车前身粘贴多层具有一定厚度的布基胶带，作为撞击缓冲层，起到一定防护的作用。在日常调试中，也只需要更换磨损的胶带层即可。主板与遥控接收器采用杜邦线连接，为了防止接触不良问题，也进行了粘胶处理。

第三章 硬件电路设计说明

本届竞赛模型车系统基于英飞凌 TC264 微控制器进行整体架构设计。硬件电路作为智能小车姿态控制、轨迹跟踪以及电机调速的物理载体，其稳定性、抗干扰能力与供电效率直接决定了小车高动态运动下的整车性能上限。

本队针对单车自平衡系统高频扰动、大电流突变及信号易受电磁干扰等核心痛点，重新规划并重构了整车硬件电路拓扑。本章将详细介绍智能车的总体设计原则、主控与感知电路、电源管理与保护电路、三相无刷电机驱动电路、外设交互接口以及 PCB 布局布线与 EMC 抗干扰优化设计。

3.1 总体设计原则与硬件架构拓扑

3.1.1 总体设计原则

为确保模型车在高速循迹与自平衡姿态调整过程中的高稳定与高响应，本系统硬件电路设计遵循以下核心原则：

1. 高可靠性与强抗干扰：严格进行数字电路与模拟电路隔离、大功率驱动回路与弱电控制回路分区；关键信号线增加阻抗匹配与滤波网络，传感器接口增加机械锁扣与胶体防护。
2. 轻量化与低重心布局：合理紧凑地安排元器件分布，缩小主板与驱动板尺寸，减少重型元器件用量，降低模型车整车质心高度，以减小车体倾覆力矩。
3. 高效供电与隔离防护：采用高效率 Buck 降压拓扑替代传统线性稳压器，减少热损耗；对舵机与主控系统实行独立电源回路供电，切断高动态大电流对主控芯片的传导干扰。
4. 模块化与通用易维护性：主控板、无刷驱动板与磁编码器子板采用模块化分板设计，引出标准测试排针与防呆接口，便于日常调试与快速故障排查。

3.1.2 系统供电与信号拓扑结构

系统采用格氏 1680mAh 3S 航模锂电池组（标称电压 11.1V，满电电压 12.6V）作为唯一动力电源。电池输出经过低损耗 PMOS 防反接电路与总开关后，分为三路供电：第一路（动力回路）直接接入三相无刷电机驱动板，为功率逆变桥提供大电流动力；第二路（主板 DCDC 降压）采用 STC2450 Buck 降压电路将 11.1V~12.6V 电池电压生成

5V 电源（VCC5V）和舵机专供 5V 电源（VCCSTEER），VCC5V 再经 RT9013-33 LDO 降压为 3.3V（VCC3V3），为主控 MCU（TC264）、IMU660RA 姿态传感器及外设接口供电；第三路（驱动板 DCDC 降压）由无刷电机驱动板采用 RY9121E Buck 降压芯片产生纯净的 5V 与 3.3V 电源，独立供给驱动板控制 MCU（CYT2BL3）及栅极驱动逻辑。

3.2 智能车主控与核心感知电路设计

3.2.1 TC264 主控核心电路

智能车主控板选用英飞凌 SAK-TC264D-40F200N BC 微控制器。该芯片集成了高性能 32 位 TriCore 内核，最高运行主频达 200MHz，内置 2.5MB Flash 和 120KB RAM，具备丰富的硬件 PWM、正交解码（QEP）及多通道 SPI/ADC 资源，完全满足姿态融合解算与串级 PID 控制的实时性要求。

主控核心外围电路主要包括：电源与解耦（数字电源 VDD 与模拟电源 VDDA 通过合理的接地布局与滤波网络隔离，未额外增加磁珠以避免寄生电感引发的电源过冲与动态响应变慢，各供电引脚处密集并联 100nF 与 4.7uF 贴片解耦电容）、时钟与复位（20MHz 无源晶振搭配 20pF 匹配电容，内置 RC 复位网络及硬件复位按键）以及标准 10-pin SWD/JTAG 下载调试接口。

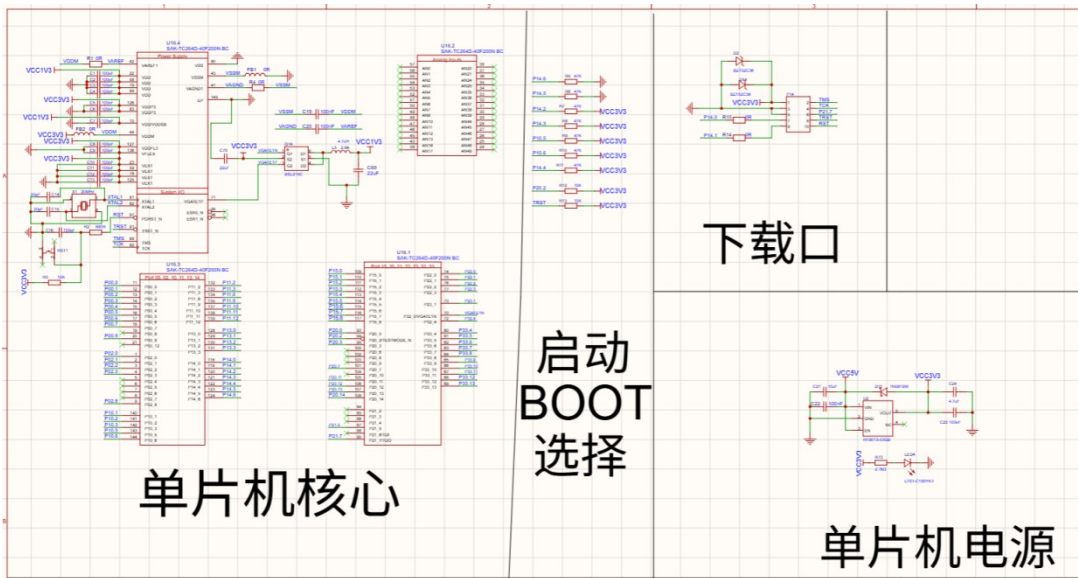


图 3.1 TC264 主控核心原理图与复位/电源电路

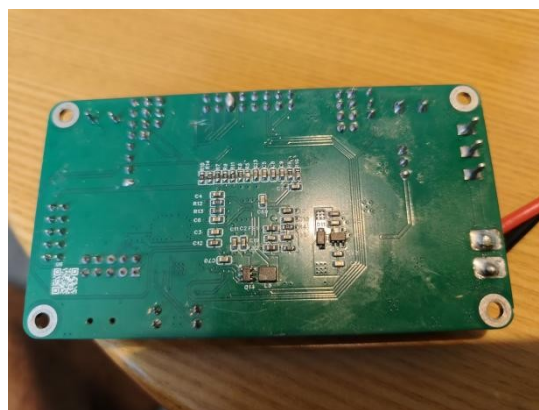
3.2.2 IMU660RA 姿态传感器电路与点胶防噪防护

对于单车倒立摆自平衡系统，车体倾角与倾角速度的测量精度直接决定了小车能否稳定维持直立状态。系统选用 SEEKFREE IMU660RA 六轴惯性测量单元，集成了高精度三轴加速度计与三轴陀螺仪，通过 SPI 高速总线与 TC264 建立通信。

由于单车行驶中存在频繁的振动与侧倾，IMU 模块被安装于紧靠车体物理重心的位置，以最小化旋转向心加速度对姿态解算的干扰。此外，在 PCB 连接引脚与模块外壳交界处实施了高强度点胶包覆防护处理，既起到了绝缘防尘作用，又有效吸收了底盘传导至传感器的机械高频振动，防止排针虚焊松动，显著提升了姿态采样的信噪比。



(a) 主板正面实物 (IMU660RA 点胶防护)



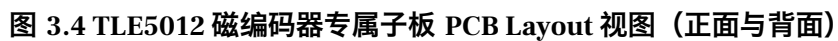
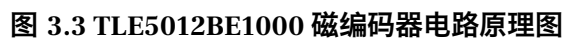
(b) 主板背面实物图

图 3.2 TC264 主控板实物照片 (正面与背面)

3.2.3 TLE5012BE1000 磁编码器电路

速度反馈对于闭环 PID 控制不可或缺。本系统摒弃了易受光线与灰尘影响的光电编码器，设计了基于 TLE5012BE1000 的巨磁阻 (GMR) 角度传感器测速子板。芯片通过感应电机转轴末端径向磁体磁场方向变化解算绝对角位置，具备无接触、高精度与抗震强等优点。

子板集成 MIC5205-3.3YM5-TR LDO 稳压芯片将输入 5V 降至 3.3V，SPI 总线串联 100Ω/470Ω 匹配电阻以减少信号反射过冲，并采用 ZX-ZH-1.5-6PWT 6-pin 带扣连接器引出，确保高速冲击下接线牢固。



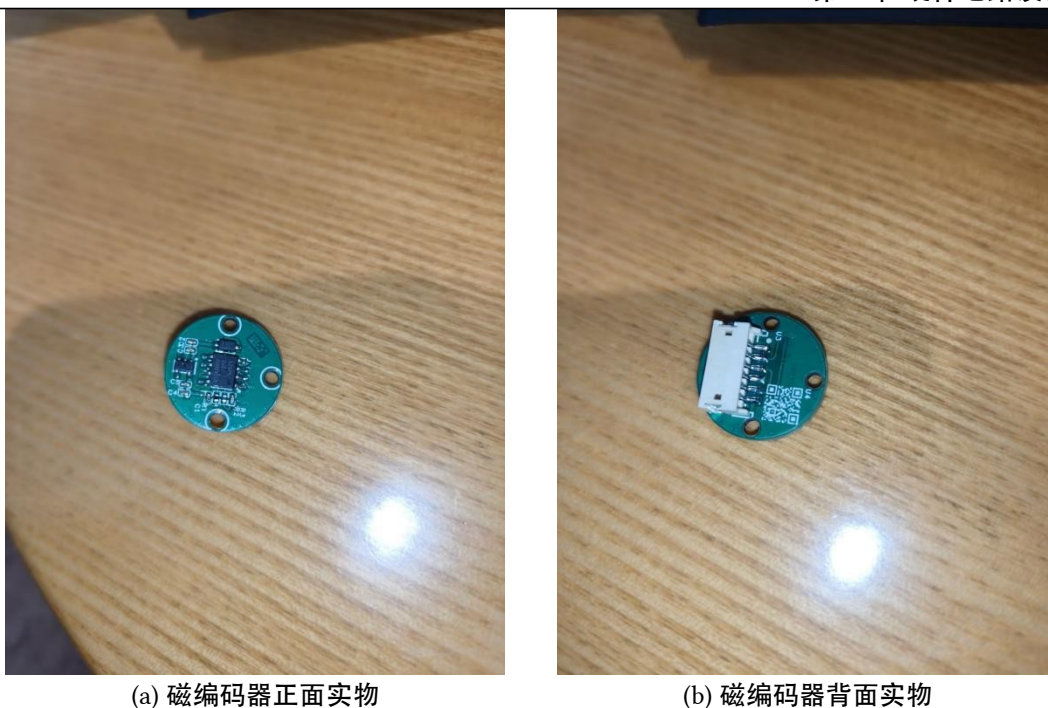


图 3.5 TLE5012 磁编码器子板实物照片（正面与背面）

3.3 电源管理与安全防护电路设计

3.3.1 多级稳压拓扑

智能车系统包含数字逻辑电路（3.3V）、传感器与外设（5V）、大功率舵机（5V 大电流）及电机驱动（11.1V 动力）。主板采用两颗高效率开关降压芯片 STC2450（工作频率高，支持大电流输出）：一路将格氏 3S 锂电池（11.1V~12.6V）降至纯净 5V（VCC5V）供板载逻辑与传感器；另一路专门为转向舵机提供独立 5V 供电（VCCSTEER），支持 5V~6V 调压以提升舵机响应，同时彻底切断舵机大电流冲击对主控系统的干扰。次级采用 RT9013-33 高 PSRR LDO 芯片二次降压生成 3.3V（VCC3V3CODE），纹波低于 15mV。

3.3.2 电池防反接与保护电路

针对日常频繁更换电池易引发的反接烧毁风险，电路设计了低损耗 P 沟道 MOSFET（HY19P03，耐压 30V，导通内阻 $R_{DS(on)} < 15m\Omega$ ）防反接电路。电源正接时 V_{gs} 产生负压驱动 PMOS 完全导通，压降不足 0.05V；反接时 PMOS 保持关断切断回路。同时栅极并联 BZT52C16 稳压管过压防护，并配置 10k Ω /100 Ω 分压网络

(BATADC) 送入 TC264 的 ADC 引脚进行电池实时电压监测与低压报警。

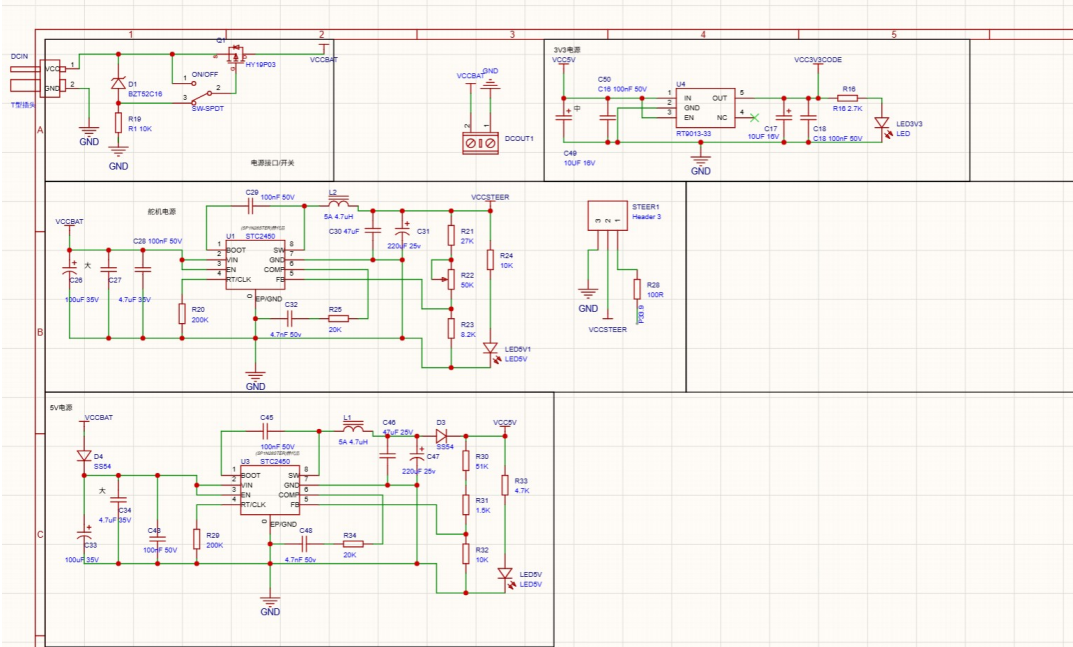


图 3.6 主板 STC2450 Buck 降压与 PMOS 防反接保护电路原理图

3.4 三相无刷电机驱动电路设计

3.4.1 CYT2BL3 无刷驱动控制主控与 RY9121E 电源

为满足赛车高速运动与大过载加速下的动力需求，团队独立设计开发了一套基于 CYT2BL3 微控制器（ARM Cortex-M4 内核）的三相无刷电机（BLDC）驱动板。驱动板集成 RY9121E Buck 同步降压芯片（支持 4.5V-40V 宽电压输入，完美兼容 3S 锂电池最高 12.6V 输出），将动力电池电压稳压降至 5V 与 3.3V，单独为 CYT2BL3 芯片及前级栅极驱动逻辑供电，实现驱动与主控电路的物理隔离。

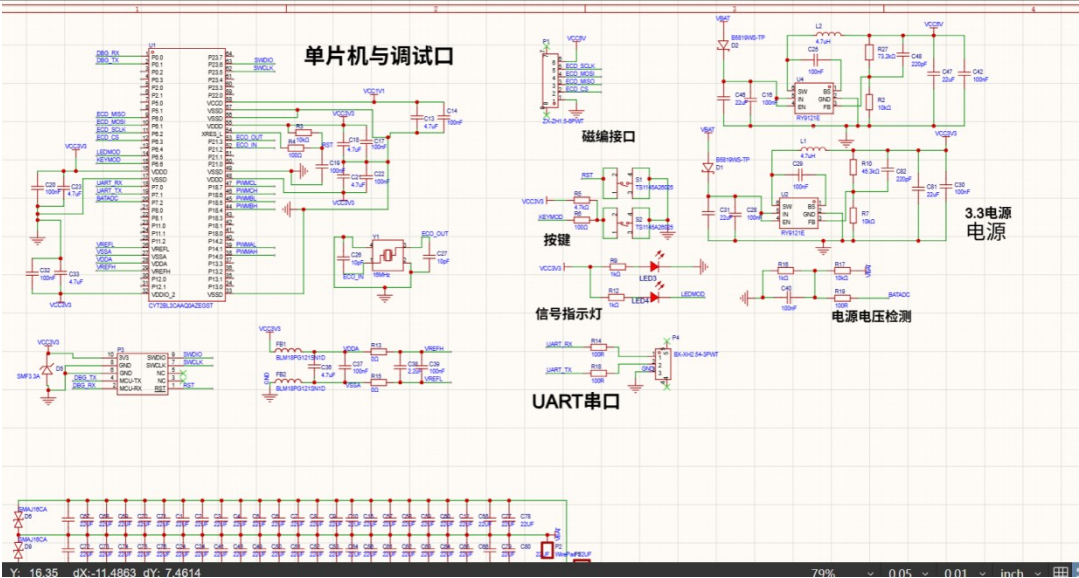


图 3.7 CYT2BL3 无刷电机驱动控制核心与电源原理图

3.4.2 三相逆变功率半桥与 PCB 散热设计

驱动板包含 VSA、VSB、VSC 三相半桥功率输出通道，每个通道由高边与低边大功率 N 沟道 MOSFET 组成。驱动板输入端密集并联了多颗 22uF/50V 低 ESR 贴片电容和大容量电解电容阵列，降低供电阻抗并吸收电机换相反电动势。三相输出大电流走线采用双面 PCB 大面积铜箔开窗并涂覆厚锡工艺，实测在 30A 持续大电流工作下驱动板温升极低，动力输出稳定线性。

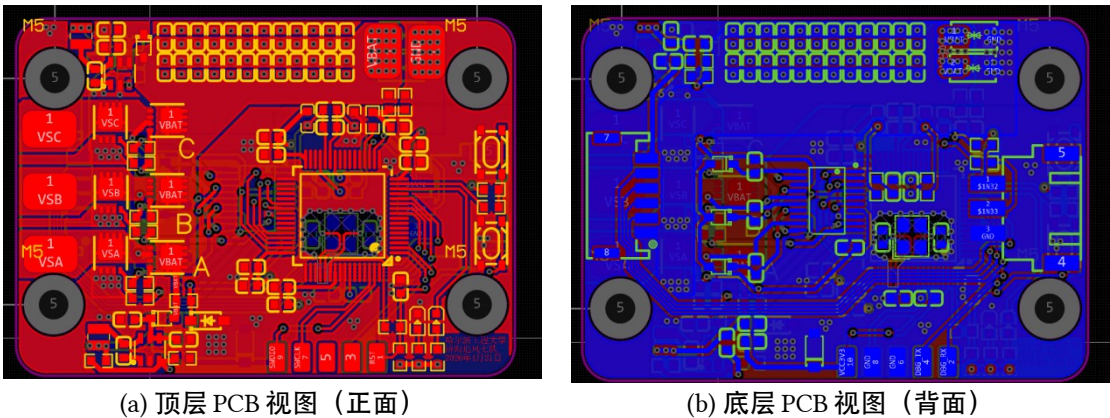
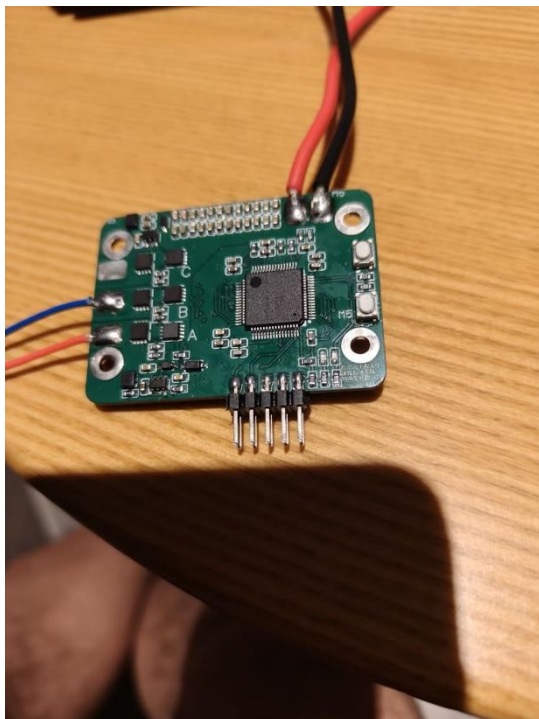
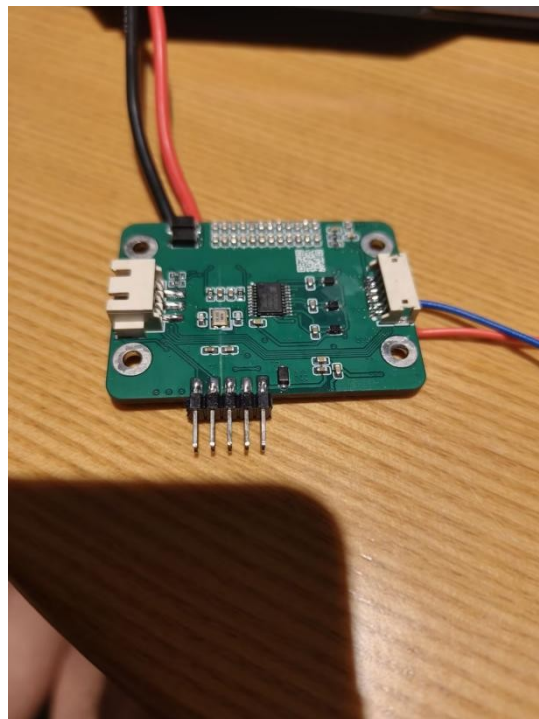


图 3.8 三相无刷驱动板 PCB Layout 视图（正面与背面）



(a) 驱动板正面实物



(b) 驱动板背面实物

图 3.9 三相无刷电机驱动板实物照片（正面与背面）

3.5 人机交互与外设扩展接口电路

3.5.1 显示屏与无线调试串口接口

主板预留 2x8P 双排针标准接口，支持直插式 TFT 屏或 IPS 显示屏，实时显示小车当前姿态角、编码器速度、PID 控制参数及赛道识别状态。同时引出 UART 蓝牙串口模块接口（包含 TXD、RXD、VCC5V、GND），支持将车身实时运行数据无线透传至上位机助手，便于在线绘制波形与参数整定。

3.5.2 五向按键与蜂鸣器提示电路

主板集成 WS-1001-ACW09 026 五向按键（上、下、左、右、中），配合限流上拉电阻网络，可通过菜单界面脱机修改控制参数与运行模式，无需频繁连接电脑烧录程序。蜂鸣器报警电路由 S8050 NPN 三极管驱动有源蜂鸣器（BEEP），用于发车确认及低压故障报警。

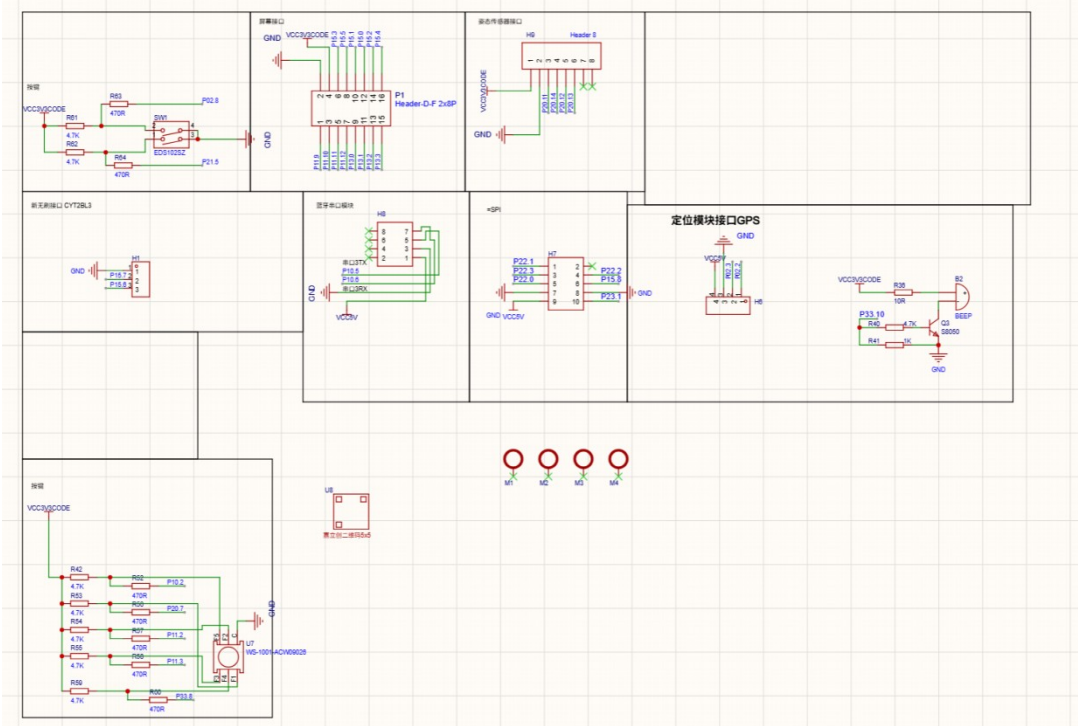
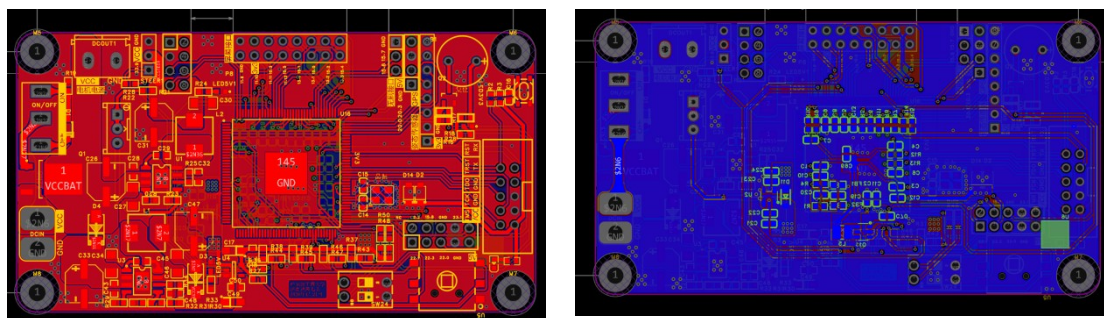


图 3.10 外设屏幕、无线串口、GPS 及五向按键/蜂鸣器电路原理图

3.6 PCB 布局布线与 EMC 抗干扰优化

智能车系统包含高频数字信号（SPI/PWM）、高压大电流动力回路（30A）及高灵敏度模拟采样信号（IMU/ADC），因此 PCB 布局布线直接决定了电路工作的稳定性。本团队在 PCB 设计中贯彻了以下优化策略：

- 1. 分区布局与合理接地：严格划分为大电流电源区、TC264 主控核心区、传感器高精信号区及驱动接口区。采用双层板结构，底层进行大面积完整铺地。数字地与模拟地通过合理的地铜走线直接单点相连，未额外使用磁珠串联，有效规避了磁珠寄生电感在大电流突变时激起的电源电压过冲与高频振荡，保证了电源的动态响应速度。
- 2. 高频解耦与过孔优化：所有芯片的 100nF 解耦电容紧贴引脚放置，过孔靠近电容焊盘打入地层以减小寄生电感。动力电源线与三相输出线采用宽线打多过孔连接，减少过孔电阻。
- 3. 机械防护与点胶工艺：所有接插件（ZH1.5/XH2.54）均具备防呆与机械卡扣功能；关键传感器（IMU660RA）及排线连接处统一实施热熔胶点胶固化防护，防止车体倾倒或碰撞时震动松脱。



(a) 主控板顶层 PCB Layout (正面)

(b) 主控板底层 PCB Layout (背面)

图 3.11 主控板 PCB Layout 双层走线与器件布局图 (正面与背面)

第四章 智能车控制软件设计说明

高效而经过长久设计的软件程序是智能车稳定平衡和循迹的基础。我们设计的智能车系统采用惯导、编码器来获取姿态和车体位置。对于平衡与循迹，要着重设计的是合理的控制算法和转向逻辑。在智能车的平衡和速度控制方面，我们使用了鲁棒性很好的经典 PID 控制算法，配合使用理论计算和实际参数补偿的办法，使智能车能够稳定前进。

4.1 路径生成与跟踪

4.1.1 路径几何生成

在慢速跟踪中，我们根据已知的场地信息，尝试用几何参数生成一条固定的平滑闭合路线。包括入场引导线、多段直线、圆弧和圆弧过渡段，目标是构造一条适合低速单车运行的平滑八字型路径。

Subject 2 generated path

--- raw line/arc path — Bezier-smoothed path • original corner

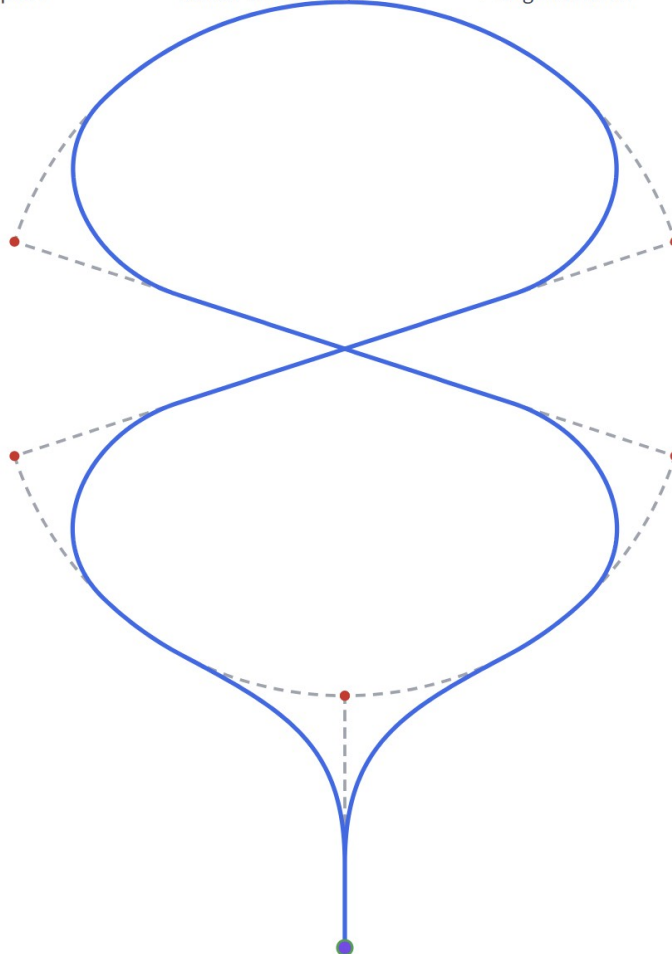


图 4.1 几何生成参考路径

路径从起点沿入口直线开始，经过第一段圆弧过渡、主圆弧、直线连接段、第二段圆弧、再次连接直线和第三段圆弧，最后通过出口过渡段和末端直线返回原点。因此，生成路径的起点和终点在同一位置，路径中间包含多个连续切换的直线段和曲线段，在路径设计上，考虑车辆沿连续曲率路线运行，不设计曲率变化过大的路线。在圆弧与直线交接处，做到过渡平滑。

直线与主圆弧之间的连接处理方式：根据主圆位置、主圆半径、转向方向、连接直线方向和候选圆角半径，求解一个同时与直线和主圆相切的过渡圆。这样可以在圆弧半径、转角和入口直线长度变化时尽量维持路径的连续性和可行性，避免生成低速下难以通过的路径。路径生成后，在路径上进行均匀踩点。

之后保存每个路径点平面坐标 $\text{slow_path_x}[i]$ 、 $\text{slow_path_y}[i]$ 和对应航向 $\text{slow_path_yaw}[i]$ 。路径点初次加入时，航向由当前点和前一个点的连线方向计算：

$$\text{yaw}_i = \text{atan2}(y_i - y_{i-1}, x_i - x_{i-1})$$

4.1.2 航向推算

对于航位推算，假设智能车在很短的时间间隔 Δt 内以速度 v 运动，且航向角为 θ （定义为相对 x 轴正方向的夹角，逆时针为正），那么这段时间内的位移增量可以分解为：

$$\Delta x = v \Delta t \cos \theta$$

$$\Delta y = v \Delta t \sin \theta$$

因此位置更新公式为：

$$x_{k+1} = x_k + v_k \Delta t \cos \theta_k$$

$$y_{k+1} = y_k + v_k \Delta t \sin \theta_k$$

其中 (x_k, y_k) 是当前时刻的位置， θ_k 是当前时刻的航向角， v_k 是当前时刻的速度大小， Δt 是采样周期。本质是把速度向量沿航向方向分解到 x 轴和 y 轴，再乘上时间得到位移，不断累加得到轨迹。航位推算的核心假设是：在每一个 Δt 内，速度和航向近似不变，或者取的是该时间段内的平均值。因此 Δt 要足够小，才能保证近似精度。速度 v 来自编码器，航向角 θ 来自陀螺仪积分结果。

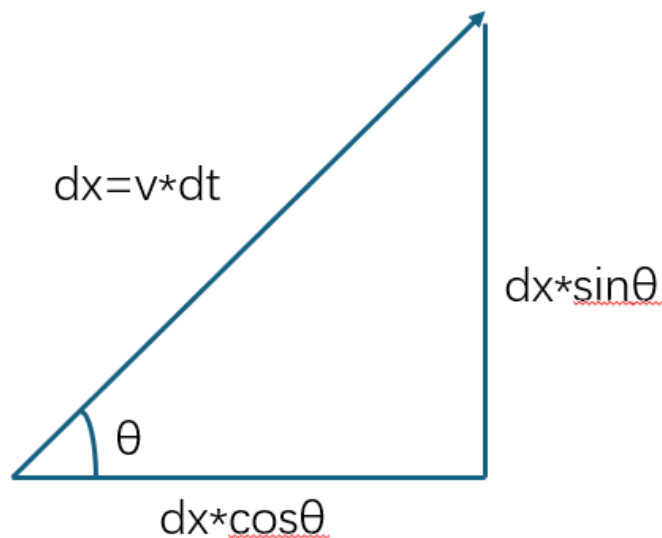


图 4.2 几何生成参考路径

由此，在程序中输入轮速 `wheel_speed` 和 IMU 输出的绝对 yaw 角，可以得到经过 `CONTROL_TIME` 的行驶距离：

$$ds = \text{wheel_speed} * \text{CONTROL_TIME}$$

再利用绝对航向将位移分解到全局平面：

$$\text{odo}_x += \text{odo}_x + ds * \cos(\text{yaw})$$

$$\text{odo}_y += \text{odo}_y + ds * \sin(\text{yaw})$$

$$\text{odo}_{\text{dist}} += \text{odo}_{\text{dist}} + |ds|$$

这里的 odo_x 和 odo_y 是车辆相对本次发车起点的全局位移。

4.1.3 跟踪点选择

对于跟踪点搜索过程，也做了一些考量。通过建立小范围的搜索窗口。窗口起点和终点分别为：

$$\text{start} = \text{near_idx} - \text{window}$$

```
end=near_idx + window
```

然后对窗口内每个路径点计算车辆位置与路径点位置的平方距离：

$$d_i^2 = (x_{\text{local}} - \text{path}_x[i])^2 + (y_{\text{local}} - \text{path}_y[i])^2$$

取距离最小的点作为当前最近点 best。但测试发现，如果前移几个索引，也就是以 best+slow_track_forward 作为前瞻点，可以更稳定运行。可以使搜索中心沿车辆行驶方向推进，在经过生成路径交叉点时不会被干扰。

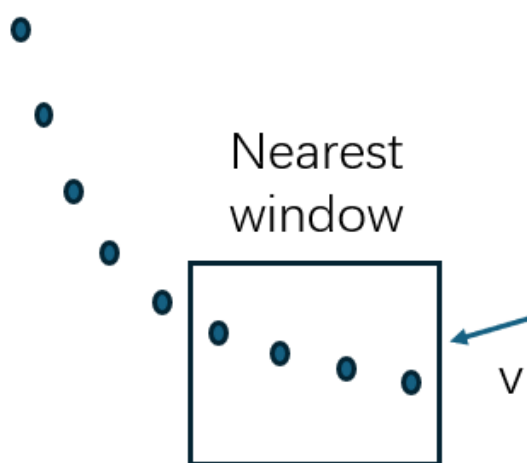


图 4.3 窗口搜索示意

4.1.4 Stanley 航向控制

Stanley 控制器是一种几何路径跟踪控制器，其核心思想是像人类驾驶员一样，通过同时关注两个关键偏差来驾驶车辆。它的控制依据有两个。一是航向偏差，即车辆当前朝向与目标路径方向之间的角度差。二是横向偏差，即车辆前轴中心到目标路径的垂直距离。将这两个误差融合，计算得到对应的前轮转向角。

Stanley 算法对应的公式为：

$$\delta = \psi_e + \arctan(k * e / v)$$

δ 是最终需要输出的前轮转向角, ψ_e 是航向偏差项, 起到追踪路径, 与路径方向对齐的作用, $\arctan(k * e / v)$ 是横向偏差的反馈项, e 代表横向偏差, v 是当前车速, k 是一个可调节的增益参数。 v 在分母上, 意味着转向角的修正量会随着车速的增加而自动减小, 从而保证了车辆在不同速度下都能稳定地跟踪路径, 避免了高速时的剧烈转向。与传统的 PID 控制器相比, 它在路径跟踪的精度和响应速度上通常表现更优。

在实际设计中, 利用轮速和 IMU 输出的绝对 yaw 来估计车辆平面位置, 并根据预先保存的路径计算横向误差和目标航向。Stanley 控制器输出目标航向 $target_yaw$, 再由上层航向控制环转换为车辆的侧倾目标, 最终完成路径跟踪。

设最近点路径航向为: $\theta = path_yaw[best]$

车辆相对最近点的位置差为: $dx = odo_x - path_x[best]$

$$dy = odo_y - path_y[best]$$

横向误差计算为: $c_error = dy * \cos\theta - dx * \sin\theta$

该公式将车辆与路径点的位置差投影到路径法向方向, 得到车辆相对路径的横向偏离量。横向误差使用当前最近点计算, 这样可以反映车辆当前实际偏离路径的程度。前瞻点只参与目标航向计算。

4.2 PID 控制算法介绍

在工程实际中, 应用最为广泛的调节器控制规律为比例、积分、微分控制, 简称 PID 控制, 又称 PID 调节。PID 控制器问世至今已有近 70 年历史, 它以其结构简单、稳定性好、工作可靠、调整方便而成为工业控制的主要技术之一。当被控对象的结构和参数不能完全掌握, 或得不到精确的数学模型时, 控制理论的其它技术难以采用时, 系统控制器的结构和参数必须依靠经验和现场调试来确定,

这时应用 PID 控制技术最为方便。即当我们不完全了解一个系统和被控对象，或不能通过有效的测量手段来获得系统参数时，最适合用 PID 控制技术。PID 控制，实际中也有 PI 和 PD 控制。

PID 控制器是一种线性控制器，它根据给定值与实际输出值构成控制偏差。将偏差的比例(P)、积分(I)和微分(D)通过线性组合构成控制量，对被控对象进行控制，故称 PID 控制器，原理框图如图 4.4 所示。

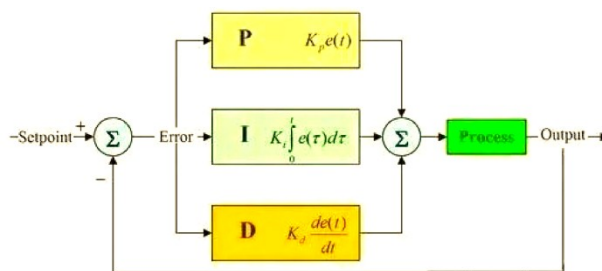


图 4.4 PID 原理框图

在计算机控制系统中，使用的是数字 PID 控制器，控制规律为：

$$e(k) = r(k) - c(k) \quad (\text{公式 4.4})$$

$$u(k) = K_p \left\{ e(k) + \frac{T}{T_i} \sum_{j=0}^k e(j) + \frac{T}{T_d} [e(k) - e(k-1)] \right\} \quad (\text{公式 4.5})$$

式中

k ——采样序号， $k = 0, 1, 2, \dots$ ； $r(k)$ ——第 k 次给定值；

$c(k)$ ——第 k 次实际输出值；

$u(k)$ ——第 k 次输出控制量；

$e(k)$ ——第 k 次偏差；

$e(k-1)$ ——第 $k-1$ 次偏差；

K_P ——比例系数；

T_I ——积分时间常数；

T_D ——微分时间常数；

T ——采样周期。

简单说来，PID 控制器各校正环节的作用如下：

比例环节：对当前误差作出即时反应，是最基本的控制作用。增大 K_P 可以加快系统响应、减小稳态误差，但过大会导致超调、振荡甚至不稳定。比例环节的控制是以当前与设定值的误差大小作为控制量的。单独使用比例控制时，对于许多被控对象会存在稳态误差，仅使用系统误差作为控制量是无法使输出完全达到设定值的。因此比例环节主要决定系统的响应强度和带宽。

积分环节：对误差进行时间累积，只要误差不为零，积分项就会持续变化，直到误差被消除。积分作用的强弱取决于积分时间常数，越大，积分作用越弱，反之则越强。

微分环节：起到预测偏差信号的变化趋势的作用，当误差快速增大时，微分项会产生反向控制作用来抑制超调，当误差开始减小时，会提供适当的阻尼，改善系统动态性能。

数字 PID 控制算法通常分为位置式 PID 控制算法和增量式 PID 控制算法。

4.2.1 位置式 PID

公式(4.5)为位置式 PID 控制算法。

位置式算法的输出对应于执行机构的实际位置，例如阀门开度、电机电压或加热功率。由于积分项需要对历史误差进行累加，计算时需要保存所有历史误差值，计算量较大，而且一旦误差长期存在，积分项会持续增大，容易造成积分饱和，使系统出现大幅超调甚至失控。因此使用位置式 PID 时，通常需要配合积分限幅、积分分离或抗积分饱和措施，同时手动/自动切换时还需进行无扰动处理，否则容易产生控制量突变。

位置式 PID 控制算法的缺点是：由于全量输出，所以每次输出均与过去的状态有关，计算时要对过去 $e(k)$ 进行累加，计算机工作量大；而且因为计算机输出的 $u(k)$ 对应的是执行机构的实际位置，如计算机出现故障， $u(k)$ 的大幅度变化，会引起执行机构位置的大幅度变化，这种情况往往是生产实践中不允许的，在某些场合，还可能造成严重的生产事故。因而产生了增量式 PID 控制的控制算法，所谓增量式 PID 是指数字控制器的输出只是控制量的增量 $\Delta u(k)$ 。

4.2.2 增量式 PID

增量式 PID 输出的是控制量的变化量。可由式(4.5)推导出提供增量的 PID 控制算式。由式(4.5)可以推出式(4.6)，式(4.5)减去式(4.6)可得式(4.4)。

$$u(k-1) = K_p \left\{ e(k-1) + \frac{T}{T_I} \sum_{j=0}^{k-1} e(j) + \frac{T}{T_D} [e(k-1) - e(k-2)] \right\} \quad \text{式(4.6)}$$

$$\Delta u(k) = K_p \left\{ [e(k) - e(k-1)] + \frac{T}{T_I} e(k) + \frac{T_D}{T} [e(k) - 2e(k-1) + e(k-2)] \right\}$$

$$\Delta u(k) = K_p \Delta e(k) + K_I e(k) + K_D [\Delta e(k) - \Delta e(k-1)] \quad \text{式(4.7)}$$

式中 $e(k) = e(k) - e(k-1)$, $K_I = K_p * T / T_I$, $K_D = K_p * T_D / T$

公式(4.7)称为增量式 PID 控制算法，可以看出由于一般计算机控制系统采用恒定的采样周期 T ，一旦确定了 K_P 、 K_I 、 K_D ，只要使用前后三次测量值的偏差，即可由式(4.7)求出控制增量。

增量式 PID 不需要累积全部历史误差，只用到最近几次误差值，计算简单，存储量小；输出增量可以方便地进行限幅，抗积分饱和能力较强；手动/自动切换时，由于输出是增量，不会产生大幅跳变，无扰动切换更容易实现。它的缺点是需要执行机构具备积分保持能力，如果执行机构丢失增量脉冲，可能造成位置漂移。

4.2.3PID 参数整定

PID 参数整定是确定比例、积分、微分系数以使控制系统达到期望动态和稳态性能的过程。通常先只调比例参数，从小到大慢慢加，看系统响应是否快速、有没有震荡；再加入积分作用来消除余差，但要注意积分太强容易引起超调或振荡；最后如果系统还不太稳定或响应不够平滑，可以适当加入微分作用来抑制波动。实际整定时往往靠经验试凑，观察车辆打弯和倾倒情况，或使用 UART 串口发送数据到计算机，查看输出曲线，然后对参数进行调整。直到系统既不会反应太慢，也不会来回震荡，就能得到一组比较合适的参数。由于单车很不稳定，要建立精确的智能车运动状态模型和控制模型有一定难度，而且我们对车身机械结构经常进行修正，模型参数变化较为频繁，理论计算整定法可操作性不强，最终我们采用了工程整定方法。此外，我们先后实验了几种动态改变 PID 参数的控制方法。

4.3 串级 PID 的设计

(1) 串级 PID 控制是一种将几个 PID 控制器按内外环串联结构组织起来的控制策略，在姿态控制中通常以角度环作为外环、角速度环作为内环。外环角度环接收目标角度与实际测量角度之间的偏差，经过 PID 运算后输出一个期望角速度指令，内环角速度环将期望角速度与陀螺仪测得的实际角速度进行比较，得到角速度误差，再经过 PID 运算产生最终的控制量，例如电机力矩、舵机输出或 PWM 占空比，从而驱动被控对象。

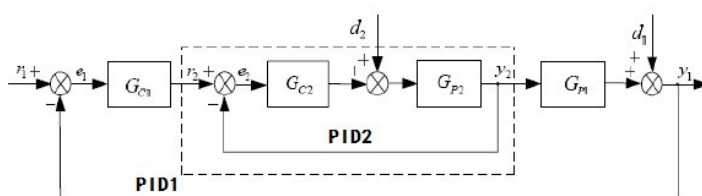


图 4.5 串级 PID 原理框图

(2) 之所以引入角速度内环，是因为角速度是角度的微分，能够更早、更快地反映姿态变化的趋势，内环可以在角度尚未出现明显偏差之前就感知并抑制扰动，而不必等到角度误差积累到一定程度才由外环纠正。从控制量到角速度通常是一个响应较快的环节，而角速度积分后才得到角度，属于相对较慢的环节；串级控制先对快速的内环进行闭环，使角速度响应加快、阻尼增大，从而让外环角度环的控制难度明显降低。

(3) 串级 PID 的主要优势体现在以下几个方面：抗扰动能力显著增强，外部风力、负载变化、碰撞等扰动作用在系统上时往往首先引起角速度波动，内环能够以较高带宽迅速抑制这种波动，防止扰动进一步累积为角度偏差，大大减轻外环的调节负担。此外，角速度环相当于为系统增加了阻尼，能够有效抑制角度响应过程中的超调和振荡，使姿态调整更加平稳，同时也增大了响应速度。

(4) 在设计控制环结构的时候，为了达到跟踪的目的，将航向环设置为外环。行驶过程中的航向误差作为输入，输出目标倾斜角，然后输入进角度和角速度的串级结构 PID。但需要注意的是，由于角速度环比较的是陀螺仪得到的角速度，但陀螺仪原始输出中往往叠加着多种噪声和干扰，不能直接当作理想的角速度反馈信号使用。一般需要对陀螺仪信号进行滤波处理，例如一阶低通滤波，同时也要保留有效的角速度动态信息，也要抑制高频噪声。但这也引入了延迟。在实际

调试时，我们发现去除角速度环可以达到更高的速度上限，将此作为一种可行的控制方案。

4.4 Mahony 算法

Mahony 算法是一种广泛应用于无人机、机器人和航模等嵌入式系统中的姿态估计算法。它的核心思想基于互补滤波，利用陀螺仪在短时间内动态响应好、积分能获得姿态变化的特点，同时利用加速度计和磁力计在长时间内能提供准确的重力方向和地磁方向参考的特点，将两者进行互补融合。陀螺仪积分得到的姿态高频特性好，但会随时间漂移；加速度计和磁力计低频精度高，但容易受振动、运动加速度和磁干扰影响。Mahony 算法通过一个 PI 控制器将这两类信息结合起来，从而得到既响应迅速又长期稳定的姿态估计。

Mahony 算法的主要优点是计算量小、实现简单，非常适合计算资源有限的嵌入式平台。它不要求解复杂的雅可比矩阵或维护协方差矩阵，相比卡尔曼滤波类算法更加轻量。同时，通过调整 PI 控制器的比例增益和积分增益，可以灵活地控制加速度计/磁力计对姿态修正的权重以及陀螺仪零偏估计的速度。在大多数平稳飞行或低速运动场景下，Mahony 算法能够提供满足控制需求的姿态精度，因此被大量开源飞控项目和商用姿态模块所采用。

Mahony 算法也有一定的局限性，例如，当智能车持续转弯、加速时，加速度计测得的重力方向常常受到外部加速度的污染，该算法会将此视为实际的重力方向处理，姿态解算会得到错误的姿态结果。此外，偏航角实际上仍然是陀螺仪积分得到的，如果不做处理，角度仍然会有比较明显的漂移现象。因此考虑适时切换 Mahony 的 KP 增益，减小加速度计大误差带来的干扰。在实际调试过程中，我们发现这样做可以明显改善高速过弯时的稳定性。

第五章 开发工具、制作、安装、调试过程说明

5.1 开发工具

程序开放在英飞凌 ADS 下进行，英飞凌 ADS 是用于 TC264 微处理器开发的一个集成开发环境。具有入门容易、使用方便和代码紧凑等特点。

5.2 调试过程简介

在本次智能车调试中，首先确定舵机中值，确保直立发车，通过静止几秒对陀螺仪采样，取变化总和的平均值，以减小零偏影响；在串级 PID 调试中，先固定 K_i 为 0，通过逐步调节 K_p 找到反应灵敏但不抖动的临界值，再调 K_d 平滑过弯，最后引入微小的 K_i 补偿累积偏差，最终确定 K_p 、 K_i 、 K_d 的值，使小车在直道平稳、弯道快速修正且不冲出赛道；在陀螺仪部分，精细调整代码中涉及陀螺仪部分的相关防止急转弯数据饱和，提高 S 弯和急弯的通过率，整体提升了循迹精度、过弯速度与运行稳定性。

调试各部分功能时，会记录一些关键的图像，如下图 5.6~5.9 所示。



图 5.6 IMU 静止时的稳定性测试

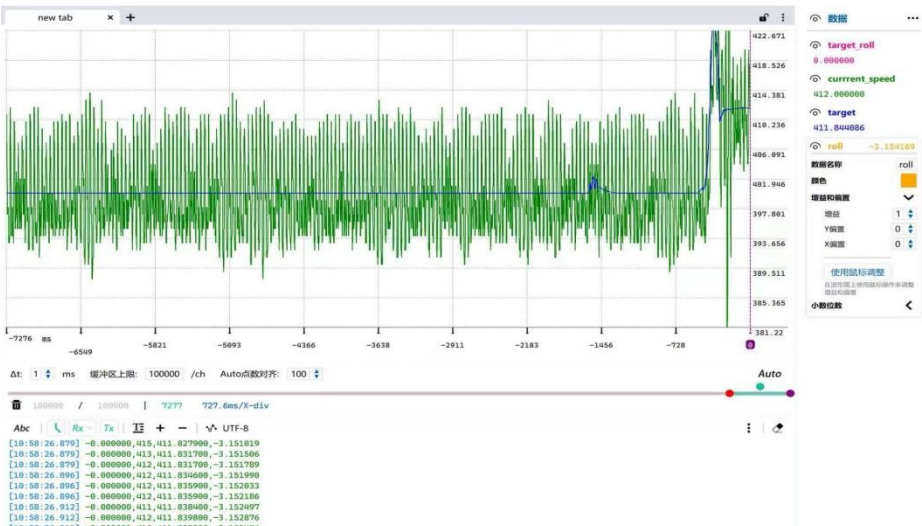


图 5.7 编码器获取速度异常时的情况

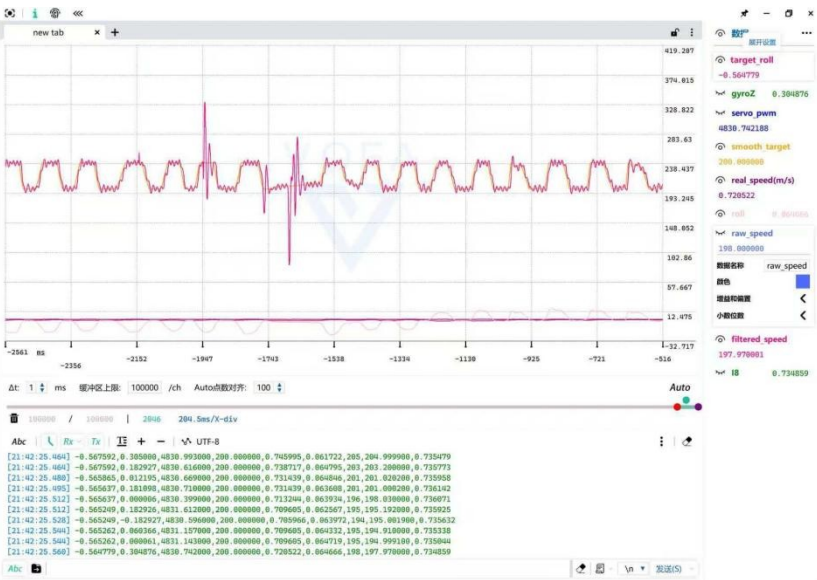


图 5.8 观察目标速度跟踪情况

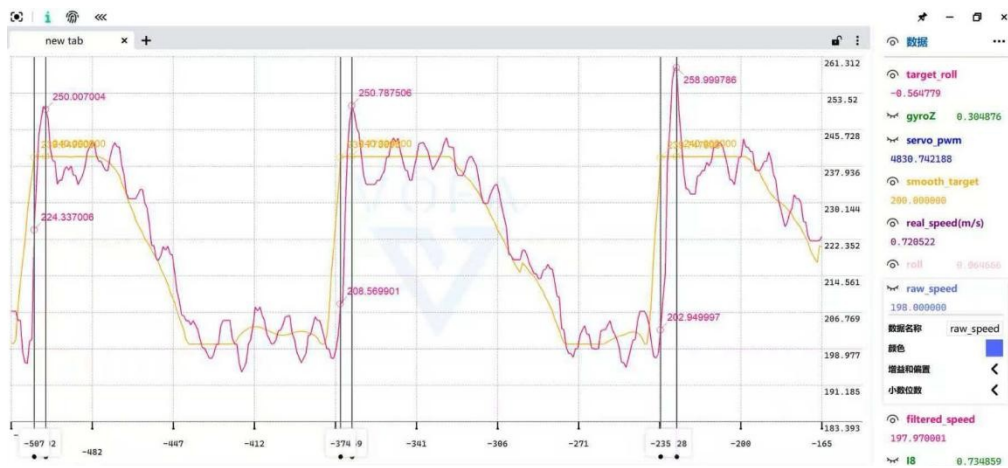


图 5.9 测量速度跟踪指标

结 论

自报名参加全国大学生智能汽车竞赛以来，我们小组成员从方案调研、机械搭建、电路调试到算法编写逐步推进，经历了多次尝试与改进，最终完成了预期目标，形成了现在这套设计方案。

在本技术报告中，我们主要总结了备赛过程中的基本思路，涵盖机械结构、硬件电路以及控制算法的设计。机械方面，我们重点分析了整车质量分布对操控性的影响，通过调整质心位置、悬挂固定等方式，提升了车辆的稳定性和灵活性。电路方面，我们采用模块化设计思路，分别对主控最小系统、电源管理、电机驱动和传感器接口进行了多方案设计与实验对比，最终确定了报告中介绍的硬件方案。软件方面，我们使用 C 语言编写程序，依托比赛推荐的开发环境进行调试，经过持续讨论和优化，形成了一套较为稳定且便于移植的代码框架。在竞速控制上，我们结合赛道元素实时调整目标车速，并优化控制策略。通过弯道减速、直道加速以及坡道平稳控制，力求在保证稳定的前提下获得最短比赛时间。

在数月的备赛过程中，我们在场地、设备和经费等方面得到了学校和学院的大力支持，在此衷心感谢一直关心和帮助我们的领导、指导老师以及学长，也感谢大赛组委会提供了这样一个锻炼与交流的平台。

如今，全国总决赛即将到来，经过长时间的准备和分区赛的锻炼，我们有信心在比赛中发挥出应有水平。虽然我们的经验和知识仍有不足，但这份技术报告凝聚了小组每一位成员的付出与智慧，是我们共同努力的见证。这段经历所带来的成长与收获，将成为我们大学生活中最珍贵的回忆。

附录：程序源代码

```
void balance_steering_loop(void)
{
    float target_yaw_error = normalize_angle(target_yaw -
target_yaw_smooth);

    uint8_t ramp_crest =
        (course_mode == COURSE_SUBJECT3 &&
        track_ramp_phase == TRACK_RAMP_CREST) ? 1u : 0u;

    uint8_t ramp_downhill =
        (course_mode == COURSE_SUBJECT3 &&
        track_ramp_phase == TRACK_RAMP_DOWNHILL) ? 1u : 0u;

    uint8_t ramp_descent_protect = (ramp_crest || ramp_downhill) ? 1u : 0u;

    if(target_yaw_error > heading_step)
        target_yaw_smooth = normalize_angle(target_yaw_smooth +
heading_step);

    else if(target_yaw_error < -heading_step)
        target_yaw_smooth = normalize_angle(target_yaw_smooth -
heading_step);

    else
        target_yaw_smooth = target_yaw;
```

// 统一采用“目标 - 当前”：右转目标 yaw 为负，输出的右倾目标角也为负

```
float yaw_error = normalize_angle(target_yaw_smooth - yaw);
```

```
/*
```

* 坡顶法向支撑变小，下坡又会放大转向动作。这里只限制外层要求的

* 航向误差和目标倾角，中环/内环仍完整工作以维持车辆站立。

```
*/
```

```
if(ramp_crest)
```

```
{
```

```
/* 坡顶不发出任何转向倾角要求，中环/内环只负责保持直立。 */
```

```
yaw_error = 0.0f;
```

```
steering_pid.integral = 0.0f;
```

```
}
```

```
else if(ramp_downhill)
```

```
{
```

```
if(yaw_error > RAMP_DOWNHILL_YAW_ERROR_LIMIT)
```

```
yaw_error = RAMP_DOWNHILL_YAW_ERROR_LIMIT;
```

```
else if(yaw_error < -RAMP_DOWNHILL_YAW_ERROR_LIMIT)
```

```
yaw_error = -RAMP_DOWNHILL_YAW_ERROR_LIMIT;
```

```
/* 下坡不允许外环积分累积成持续的大转向要求。 */
```

```
steering_pid.integral = 0.0f;
```

```
}
```

```
target_angle = pid_calculate_by_error(&steering_pid, yaw_error);

if(ramp_crest)
{
    target_angle = 0.0f;
}

else if(ramp_downhill)
{
    if(target_angle > RAMP_DOWNHILL_TARGET_ANGLE_MAX)
        target_angle = RAMP_DOWNHILL_TARGET_ANGLE_MAX;
    else if(target_angle < -RAMP_DOWNHILL_TARGET_ANGLE_MAX)
        target_angle = -RAMP_DOWNHILL_TARGET_ANGLE_MAX;
}

/*
 * 坡顶/下坡一旦横向倾角或倾倒角速度已经偏大，继续追航向只会
 * 加速摔倒；立即撤掉外环转向要求，让平衡两环优先回正。
 */

if(ramp_descent_protect &&
    (fabsf(pitch_balance) > RAMP_RECOVERY_LEAN_DEG ||
     fabsf(gyro_y_rate) > RAMP_RECOVERY_GYRO_DPS))
{
    target_angle = 0.0f;
}
}
```

```
void course_control_2ms(void)
{
    if(!course_is_enabled())
    {
        servo_output = 0.0f;
        servo_set(mid);
        return;
    }

    if(course_is_slow())
    {
        if(!car_is_armed())
        {
            slow_control_reset();
            servo_output = 0.0f;
            servo_set(mid);
            return;
        }

        pitch_balance = pitch - slow_balance_zero_angle;
        pid_set_target(&slow_angle_pid, target_angle);
        {
            float angle_output = slow_angle_pid_calculate(pitch_balance);
```

```
/* 一阶低通： alpha 越小滤波越强， alpha=1 时等同于不滤波。 */
servo_output += slow_servo_filter_alpha *
    (angle_output - servo_output);
}

servo_set((uint32_t)((int32_t)mid + (int32_t)servo_output));
}

else
{
    balance_gyro_loop();
}
}

void course_control_10ms(void)
{
    if(!course_is_enabled())
        return;

    if(!course_is_slow())
    {
        if(course_mode == COURSE_SUBJECT3 && track_playing)
            subject3_gain_schedule_update();

        balance_angle_loop();

        return;
    }
}
```

```
target_yaw_smooth = target_yaw;

target_angle = pid_calculate_by_error(
    &slow_steering_pid, normalize_angle(target_yaw_smooth - yaw));
}

void course_control_20ms(void)
{
    if(course_is_enabled() && !course_is_slow())
        balance_steering_loop();
}
```